

Contents

Les Types de Collections en Python	3
1 Les Listes	4
Création d'une liste	4
Accès aux éléments	4
Slicing	4
Modification	5
Compréhensions de listes	5
2 Les Tuples	5
Création	5
Immutabilité	6
Unpacking	6
3 Les Dictionnaires	6
Création	6
Accès et modification	7
Méthodes keys, values, items	7
Parcours	7
Méthode sécurisée get()	7
4 Les Ensembles	8
Création	8
Opérations ensemblistes	8
Méthodes — discard vs remove	8
5 Les Tableaux Typés (array)	9
Création	9
Contrainte de type	9
6 Mutabilité des collections	9
7 Comparaison synthétique	10
8 Mini-exercices de production	10
Exercice 1 — Prédiction	10
Exercice 2 — Production	11
Exercice 3 — Débogage	11
Synthèse	12
20 Questions Intelligentes – Collections Python	12
1) Pourquoi modifier une liste copiée peut aussi modifier l'originale ?	12

Énoncé	12
Solution	13
2) Comment créer une vraie copie indépendante d'une liste ?	13
Énoncé	13
Solution	13
3) Pourquoi une liste contenant une sous-liste reste liée malgré copy() ?	14
Énoncé	14
Solution	14
4) Comment faire une vraie copie profonde ?	14
Énoncé	14
Solution	14
5) Pourquoi un tuple peut contenir une liste modifiable ?	15
Énoncé	15
Solution	15
6) Pourquoi utiliser un tuple comme clé de dictionnaire ?	15
Énoncé	15
Solution	15
7) Pourquoi une liste ne peut pas être clé ?	16
Énoncé	16
Solution	16
8) Pourquoi set supprime les doublons ?	16
Énoncé	16
Solution	16
9) Pourquoi l'ordre change dans un set ?	17
Énoncé	17
Solution	17
10) Comment trier un dictionnaire par valeur ?	17
Énoncé	17
Solution	17
11) Pourquoi get() est plus sûr que l'indexation directe ?	18
Énoncé	18
Solution	18
12) Différence entre pop() et remove() dans une liste ?	18
Énoncé	18
Solution	18

13) Pourquoi utiliser enumerate ?	19
Énoncé	19
Solution	19
14) Différence entre append et extend ?	19
Énoncé	19
Solution	19
15) Pourquoi utiliser frozenset ?	20
Énoncé	20
Solution	20
16) Comment fusionner deux dictionnaires proprement ?	20
Énoncé	20
Solution	20
17) Pourquoi sorted() ne modifie pas la liste ?	21
Énoncé	21
Solution	21
18) Comment compter les occurrences efficacement ?	21
Énoncé	21
Solution	21
19) Pourquoi utiliser defaultdict ?	22
Énoncé	22
Solution	22
20) Cas complet : Gestion d'inventaire	22
Énoncé	22
Solution	22

Les Types de Collections en Python

Le support est une propriété de **Bechir Bejaoui**

Les collections permettent de regrouper, organiser et manipuler plusieurs données au sein d'une même structure. Chaque type possède des propriétés spécifiques adaptées à des usages précis en développement professionnel.

Les principales collections en Python sont :

- Les listes
- Les tuples
- Les dictionnaires

- Les ensembles
 - Les tableaux typés
-

1 Les Listes

Une liste est une séquence ordonnée et modifiable d'éléments. Elle accepte des types hétérogènes et autorise la duplication.

Création d'une liste

```
"""
Création d'une liste simple et d'une liste hétérogène.
"""

nombres = [10, 20, 30, 40]
mixte = [1, "Python", True, 3.14]

print(nombres)
print(mixte)
```

Accès aux éléments

```
"""
Indexation positive et négative.
"""

print(nombres[0])      # Premier élément
print(nombres[-1])     # Dernier élément

nested = [1, [2, 3], 4]
print(nested[1][0])    # Accès imbriqué
```

Slicing

```
"""
Extraction de portions avec pas personnalisé.
"""

liste = [1, 2, 3, 4, 5, 6, 7]

print(liste[1:4])      # Extraction
print(liste[::2])      # Un élément sur deux
print(liste[::-1])     # Inversion
```

Modification

```
"""
Modification, ajout, insertion et suppression d'éléments.
"""

liste = [1, 2, 3]

liste[1] = 200      # Modifier
liste.append(4)     # Ajouter en fin
liste.insert(0, 0)  # Insérer à l'index 0
liste.remove(200)   # Supprimer par valeur

print(liste)
```

Compréhensions de listes

```
"""
Syntaxe concise pour générer des listes.
Ce mécanisme sera approfondi en Partie II.
"""

carres = [x**2 for x in range(1, 11)]
pairs = [x for x in range(20) if x % 2 == 0]

print(carres)
print(pairs)
```

2 Les Tuples

Un tuple est une séquence ordonnée mais immuable.

Il est utilisé pour représenter des données fixes ou des retours multiples.

Création

```
"""
Différentes formes de création d'un tuple.
"""

coordonnees = (10, 20)
point = 5, 6      # Sans parenthèses
unique = (100,)   # Tuple à un seul élément - virgule obligatoire
```

```
print(coordonnees)
print(unique)
```

Immutabilité

```
"""
Toute tentative de modification provoque une erreur.
"""
```

```
t = (1, 2, 3)

try:
    t[0] = 10
except TypeError as e:
    print("Erreur :", e)
```

Unpacking

```
"""
Déstructuration d'un tuple retourné par une fonction.
"""
```

```
def get_user():
    return "Alice", 30

nom, age = get_user()

print(nom)
print(age)
```

3 Les Dictionnaires

Un dictionnaire stocke des paires clé-valeur. Les clés doivent être immuables.

Création

```
"""
Création d'un dictionnaire avec types variés.
"""
```

```
user = {
    "nom": "Alice",
    "age": 30,
    "actif": True
```

```
}

print(user)
```

Accès et modification

```
"""
Lecture, modification et ajout de clés.
"""
```

```
print(user["nom"])

user["age"] = 31
user["email"] = "alice@test.com"

print(user)
```

Méthodes keys, values, items

```
"""
Accès aux clés, valeurs et paires.
Ces méthodes sont les plus utilisées en parcours.
"""

print(user.keys())    # Toutes les clés
print(user.values())  # Toutes les valeurs
print(user.items())   # Paires clé-valeur
```

Parcours

```
"""
Itération sur les paires clé-valeur.
"""
```

```
for cle, valeur in user.items():
    print(cle, "->", valeur)
```

Méthode sécurisée get()

```
"""
get() retourne une valeur par défaut si la clé est absente.
Évite un KeyError.
"""
```

```
print(user.get("adresse", "Non définie"))
```

4 Les Ensembles

Un ensemble est une collection non ordonnée et sans doublons.

Création

```
"""
Création d'un set et d'un set vide.
"""

s = {1, 2, 3, 3, 4}    # Les doublons sont automatiquement supprimés
print(s)

vide = set()           # set() et non {} qui crée un dict vide
print(vide)
```

Opérations ensemblistes

```
"""
Les quatre opérations fondamentales sur les ensembles.
"""

A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)    # Union
print(A & B)     # Intersection
print(A - B)    # Différence
print(A ^ B)    # Différence symétrique
```

Méthodes — discard vs remove

```
"""
add ajoute un élément.
remove supprime un élément - lève une exception si absent.
discard supprime un élément - silencieux si absent.
"""

s = {1, 2, 3}

s.add(10)
s.remove(2)
s.discard(99)    # Pas d'erreur même si 99 est absent

try:
    s.remove(99)
```



```
except KeyError as e:
    print("Erreur remove :", e)

print(s)
```

5 Les Tableaux Typés (array)

Les tableaux permettent de stocker des données homogènes de manière plus efficace en mémoire.

Création

```
"""
Un array n'accepte qu'un seul type de données.
Plus économe en mémoire qu'une liste pour les données numériques.
"""
```

```
from array import array

tab = array("i", [1, 2, 3, 4])    # "i" = entiers signés
print(tab)

tab.append(5)
print(tab)
```

Contrainte de type

```
"""
Toute tentative d'ajout d'un type incompatible provoque une erreur.
"""

try:
    tab.append("texte")
except TypeError as e:
    print("Erreur :", e)
```

6 Mutabilité des collections

Type	Mutable ?
list	Oui
dict	Oui

Type	Mutable ?
set	Oui
array	Oui
tuple	Non
frozenset	Non

7 Comparaison synthétique

Type	Ordonné	Modifiable	Doublons	Clés personnalisées
Liste	Oui	Oui	Oui	Non
Tuple	Oui	Non	Oui	Non
Dictionnaire	Oui*	Oui	Clés uniques	Oui
Set	Non	Oui	Non	Non
Array	Oui	Oui	Oui	Non

*Ordre d'insertion conservé depuis Python 3.7.

8 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
liste = [1, 2, 3]
copie = liste
copie.append(4)
print(liste)
print(liste is copie)
```

```
t = (1, [2, 3])
t[1].append(99)
print(t)
```

```
s = {3, 1, 2, 2, 1}
print(s)
```

Exercice 2 — Production

Écrire un programme qui :

1. Crée un dictionnaire de 3 produits avec leur prix
2. Ajoute un nouveau produit
3. Modifie le prix d'un produit existant
4. Affiche tous les produits avec leur prix via `items()`
5. Utilise `get()` pour accéder à un produit inexistant

Votre solution ici

Solution

```
"""
Gestion simple d'un catalogue produits.
"""

catalogue = {
    "Laptop": 999.99,
    "Souris": 29.90,
    "Clavier": 59.90
}

catalogue["Écran"] = 349.00
catalogue["Souris"] = 24.90

for produit, prix in catalogue.items():
    print(f"{produit} : {prix} €")

print(catalogue.get("Casque", "Produit non disponible"))
```

Exercice 3 — Débogage

Ce code contient 3 erreurs. Les identifier et les corriger.

```
t = (1, 2, 3)
t[0] = 10

d = {}
print(d["nom"])

s = {[1, 2], [3, 4]}
print(s)
```

Solution

Erreur 1 : tuple immuable → modification impossible

```

t = (1, 2, 3)
print(t[0])    # lecture seule

# Erreur 2 : clé inexistante → utiliser get()
d = {}
print(d.get("nom", "Non défini"))

# Erreur 3 : liste non hashable → ne peut pas être élément de set
s = {(1, 2), (3, 4)}    # utiliser tuples
print(s)

```

Synthèse

- Une liste est ordonnée, mutable, hétérogène
 - Un tuple est ordonné, immuable — idéal pour données fixes et retours multiples
 - Un dictionnaire stocke des paires clé-valeur — clés immuables obligatoires
 - Un set est non ordonné, sans doublons — puissant pour opérations ensemblistes
 - `discard` est plus sûr que `remove` sur un set
 - `get()` est plus sûr que l'indexation directe sur un dict
 - `copy()` est superficielle — utiliser `deepcopy()` pour structures imbriquées
-

20 Questions Intelligentes – Collections Python

Format : Énoncé → Solution complète avec démarche explicative et démonstration observable

1) Pourquoi modifier une liste copiée peut aussi modifier l'originale ?

Énoncé

Comprendre la différence entre copie superficielle et référence.

Solution

```
"""
Explication
L'affectation simple ne crée pas une nouvelle liste
mais une référence vers le même objet en mémoire.
"""

liste1 = [1, 2, 3]
liste2 = liste1    # référence vers le même objet

liste2.append(4)

print("Liste1 :", liste1)
print("Liste2 :", liste2)
print("Même objet mémoire ?", id(liste1) == id(liste2))
```

2) Comment créer une vraie copie indépendante d'une liste ?

Énoncé

Corriger le problème précédent.

Solution

```
"""
Explication
copy() crée une nouvelle liste indépendante.
Les modifications n'affectent plus l'originale.
"""

liste1 = [1, 2, 3]
liste2 = liste1.copy()

liste2.append(4)

print("Liste1 :", liste1)
print("Liste2 :", liste2)
print("Même objet mémoire ?", id(liste1) == id(liste2))
```

3) Pourquoi une liste contenant une sous-liste reste liée malgré copy() ?

Énoncé

Comprendre la limite de la copie superficielle.

Solution

```
"""
Explication
copy() duplique la structure externe uniquement.
Les sous-objets restent partagés entre les deux listes.
"""

liste1 = [[1, 2], [3, 4]]
liste2 = liste1.copy()

liste2[0].append(99)

print("Liste1 :", liste1)
print("Liste2 :", liste2)
```

4) Comment faire une vraie copie profonde ?

Énoncé

Résoudre le problème des objets imbriqués.

Solution

```
"""
Explication
deepcopy copie récursivement tous les sous-objets.
L'original n'est plus affecté.
"""

from copy import deepcopy

liste1 = [[1, 2], [3, 4]]
liste2 = deepcopy(liste1)

liste2[0].append(99)
```

```
print("Liste1 :", liste1)
print("Liste2 :", liste2)
```

5) Pourquoi un tuple peut contenir une liste modifiable ?

Énoncé

Comprendre la notion d'immuabilité structurelle.

Solution

```
"""
Explication
Le tuple est immuable dans sa structure.
Mais il peut référencer des objets mutables.
Ces objets internes restent modifiables.
"""
```

```
t = (1, [2, 3], 4)
```

```
t[1].append(99)
```

```
print(t)
```

6) Pourquoi utiliser un tuple comme clé de dictionnaire ?

Énoncé

Comprendre la contrainte des clés immuables.

Solution

```
"""
Explication
Les clés d'un dictionnaire doivent être hashables.
Un tuple est immuable donc hashable.
"""
```

```
coord = (10, 20)
```

```
points = {coord: "Position A"}

print(points[(10, 20)])
```

7) Pourquoi une liste ne peut pas être clé ?

Énoncé

Observer l'erreur.

Solution

```
"""
Explication
Une liste est mutable donc non hashable.
Python interdit son usage comme clé.
"""

try:
    d = {[1, 2]: "erreur"}
except TypeError as e:
    print("Erreur :", e)
```

8) Pourquoi set supprime les doublons ?

Énoncé

Comprendre le principe d'unicité.

Solution

```
"""
Explication
Un set est basé sur une table de hachage.
Chaque élément ne peut apparaître qu'une fois.
"""

liste = [1, 2, 2, 3, 3, 3]
s = set(liste)

print("Liste originale :", liste)
print("Set :", s)
```

9) Pourquoi l'ordre change dans un set ?

Énoncé

Comprendre que set n'est pas ordonné.

Solution

```
"""
Explication
L'ordre d'un set dépend du hachage interne.
Il n'est pas garanti et peut varier.
"""

s = {5, 1, 3, 2, 4}
print(s)
```

10) Comment trier un dictionnaire par valeur ?

Énoncé

Réaliser un tri personnalisé.

Solution

```
"""
Explication
sorted avec key=lambda permet de trier par valeur.
dict() reconstruit le résultat.
"""

d = {"a": 3, "b": 1, "c": 2}

tri = dict(sorted(d.items(), key=lambda x: x[1]))

print(tri)
```

11) Pourquoi get() est plus sûr que l'indexation directe ?

Énoncé

Éviter KeyError.

Solution

```
"""
Explication
L'accès direct lève une KeyError si la clé est absente.
get() retourne une valeur par défaut.
"""

d = {"nom": "Alice"}

print(d.get("age", "Non défini"))

try:
    print(d["age"])
except KeyError as e:
    print("Erreur :", e)
```

12) Différence entre pop() et remove() dans une liste ?

Énoncé

Comparer comportement.

Solution

```
"""
Explication
pop supprime par index et retourne la valeur.
remove supprime par valeur.
"""

liste = [10, 20, 30]

valeur = liste.pop(1)
print("Après pop :", liste, "Valeur retirée :", valeur)
```

```
liste.remove(10)
print("Après remove :", liste)
```

13) Pourquoi utiliser enumerate ?

Énoncé

Accéder index et valeur simultanément.

Solution

```
"""
Explication
enumerate retourne des paires (index, valeur).
Evite de gérer manuellement un compteur.
"""
```

```
liste = ["a", "b", "c"]

for index, valeur in enumerate(liste):
    print(index, valeur)
```

14) Différence entre append et extend ?

Énoncé

Observer le comportement.

Solution

```
"""
Explication
append ajoute un élément unique - même si c'est une liste.
extend déroule et ajoute chaque élément.
"""
```

```
liste = [1, 2, 3]
liste.append([4, 5])
print("Après append :", liste)

liste = [1, 2, 3]
```

```
liste.extend([4, 5])
print("Après extend :", liste)
```

15) Pourquoi utiliser frozenset ?

Énoncé

Utilisation comme clé ou élément immuable.

Solution

```
"""
Explication
frozenset est immuable et hashable.
Il peut être utilisé comme clé de dictionnaire.
"""

fs = frozenset([1, 2, 3])
print(fs)

d = {fs: "groupe A"}
print(d)
```

16) Comment fusionner deux dictionnaires proprement ?

Énoncé

Approche moderne.

Solution

```
"""
Explication
L'opérateur | fusionne deux dictionnaires (Python 3.9+).
En cas de clé commune, la valeur du second dict prend priorité.
"""

d1 = {"a": 1}
d2 = {"b": 2}

fusion = d1 | d2
```

```
print(fusion)
```

17) Pourquoi sorted() ne modifie pas la liste ?

Énoncé

Différence sort() vs sorted().

Solution

```
"""
Explication
sorted retourne une nouvelle liste triée.
sort modifie la liste en place.
"""

liste = [3, 1, 2]

tri_temp = sorted(liste)
print("Liste originale :", liste)
print("Tri temporaire :", tri_temp)

liste.sort()
print("Tri en place :", liste)
```

18) Comment compter les occurrences efficacement ?

Énoncé

Utiliser Counter.

Solution

```
"""
Explication
Counter crée un dictionnaire de comptage automatiquement.
Plus efficace qu'une boucle manuelle.
"""

from collections import Counter
```

```
liste = ["a", "b", "a", "c", "b", "a"]

compte = Counter(liste)

print(compte)
```

19) Pourquoi utiliser defaultdict ?

Énoncé

Éviter vérification d'existence.

Solution

```
"""
Explication
defaultdict initialise automatiquement la valeur par défaut.
Evite les vérifications manuelles d'existence de clé.
"""

from collections import defaultdict

d = defaultdict(list)

d["a"].append(1)
d["a"].append(2)

print(d)
```

20) Cas complet : Gestion d'inventaire

Énoncé

Utiliser plusieurs collections ensemble.

Solution

```
"""
Explication
Simulation d'un mini système d'inventaire.
Dictionnaire pour le stock.
"""
```

```
Liste de tuples pour les transactions.
"""

stock = {
    "Laptop": 10,
    "Mouse": 25
}

transactions = [
    ("Laptop", -2),
    ("Mouse", -5),
    ("Laptop", 5)
]

for produit, quantite in transactions:
    if produit in stock:
        stock[produit] += quantite

print("Stock final :", stock)
```